

```

MaxAvailable int NULL,
CurrUsage int NULL,
MinUsage int NULL,
MaxUsage int NULL,
InfoUnitID int NULL,
SysLogTag varchar(60),
EventLogType varchar(60),
GenericFileName VarChar(60),
SystemID int NULL
);

CREATE TABLE SystemEventsProperties
(
    ID int unsigned not null auto_increment primary key,
    SystemEventID int NULL,
    ParamName varchar(255) NULL,
    ParamValue text NULL
);

```

To confirm that the database has been successfully created, check the list of databases for the new database, Syslog, using the following command:

```
show databases;
```

Create a new MySQL user and grant permission to use the newly created database, as shown below. In our set-up, the username is *rsysloguser* and the password is *rsyslogpw*.

```

CREATE USER 'rsysloguser'@'localhost' IDENTIFIED BY
'rsyslogpw';
GRANT ALL ON Syslog.* TO 'rsysloguser'@'localhost';

```

The last step is to configure Rsyslog in both the hosts. The configuration file is located at */etc/rsyslog.conf*. To understand the structure of this file, let's have a look at the basic message processing work-flow of Rsyslog (see Figure 5). For each type of log source, we have a separate input module, e.g., *imtcp* for receiving log messages over TCP/IP, *imklog* for the kernel log, etc. Similarly, for each type of log destination, there is a separate output module, e.g., *omfwd* to forward the log to a remote host over the IP network, *ommysql* to send logs to a MySQL server, etc. To enable a particular type of input or output, we have to load the respective modules using *module(load="<module>"...)* before we create the *input(type="<input-module>"...)* and *action(type="<output-module>"...)* configuration objects to actually process logs to and from them. Some of the input and output types are natively supported by Rsyslog, so we need not load the respective modules before use, e.g., *omfwd* can be directly used in *action(...)* objects before loading them. Similarly, for some modules, loading is sufficient to get log messages from the source, *input(...)* and objects are not required, e.g., *imklog*.

Next, a set of rules needs to be specified to match log

messages against a particular input type. Finally, on matching, a set of log messages will be redirected to output via *action(...)* objects. Now let's configure Rsyslog on the first host, Ubuntu1404lts1, as follows:

```

sudo gedit /etc/rsyslog.conf
/etc/rsyslog.conf on Ubuntu1404lts1
#### MODULES ####
# Load modules
module(load="imuxsock") # For local system log, input()
optional
module(load="imklog") # For kernel log, input() not required
#### RULES ####
# Rule to send all local log messages to Ubuntu1404lts2

*. * action(
    type="omfwd" # output module to be used
    Target="ubuntu1404lts2" # ip also can be specified
    Port="514"
    Protocol="tcp"
)

```

Host Ubuntu1404lts1 has to do the following:

1. Receive its own system and kernel log messages, and...
2. Send them to Ubuntu1404lts2 over TCP/IP. (



Note: Make sure the host name Ubuntu1404lts2 is reachable from Ubuntu1404lts1 and vice versa. Either add IP-hostname mapping in the */etc/hosts* file on both the systems, or directly use the IP address in the Rsyslog configuration file instead of the hostname.

Let's move to the configuration on the second host, Ubuntu1404lts2, by issuing the following code:

```

/etc/rsyslog.conf on Ubuntu1404lts2
#### MODULES ####

# Load modules
module(load="imtcp") # For TCP input support
module(load="ommysql") # For MySQL output support

#### RULES ####

# Create a ruleset
ruleset(name="rsRemoteTCP") {
    *. * action(
        type="ommysql"
        server="localhost"
        serverport="4403"
        db="Syslog"
        uid="rsysloguser"
        pwd="rsyslogpw"
    )
}

```